

Assignment-1 : Transformers from Scratch, Architectural Variants, and Byte Latent Transformers

Bhavani Shankar Y

Roll Number: 2026900001

Advanced Natural Language Processing
y.shankar@students.iiit.ac.in

1 Introduction

1.1 Attention

Multi-Head Attention (MHA) [C1, C2, C4, C5]:

MHA projects queries, keys, and values into multiple heads that compute attention independently before their outputs are concatenated.

Grouped-Query Attention (GQA) [C3]: GQA uses fewer key and value heads than query heads, allowing groups of queries to share key and value representations.

The custom MHA and GQA implementations are available in the project attention module (Shankar, 2026b).

1.2 Positional Encoding

Sinusoidal Positional Encoding [C1, C3, C4, C5]:

Fixed sine and cosine vectors are added to the embeddings to represent sequence positions.

Rotary Positional Encoding (RoPE) [C2]:

RoPE applies position-dependent rotations to queries and keys, allowing relative positions to influence attention scores.

The corresponding implementations are available in the positional and attention modules (Shankar, 2026c,d).

1.3 Normalization

Layer Normalization [C1, C2, C3, C5]: LayerNorm normalizes each representation using its feature-wise mean and variance, followed by learnable scale and bias parameters.

RMS Normalization [C4]: RMSNorm scales each representation by its root mean square without subtracting the mean.

Both methods are implemented in the project normalization module (Shankar, 2026a).

1.4 Input Embeddings

BPE-Based Embeddings [C1–C4]: BPE learns variable-length subword units by repeatedly merg-

Statistic	Ciphertext	Plaintext
Number of sequences	5,000	5,000
Empty sequences	0	0
Unique characters	2	52
Minimum sequence length	169	22
Maximum sequence length	21,361	2,671
Mean sequence length	4,781.85	598.61
Median sequence length	4,429.00	554.50
75th percentile length	6,593.00	825.00
90th percentile length	8,681.80	1,086.10

Table 1: Summary statistics for the aligned ciphertext and plaintext sequences. Sequence lengths exclude the trailing newline character.

ing frequent adjacent pairs and maps the resulting token IDs to dense embeddings.

BLT-Style Embeddings [C5]: The simplified BLT-style approach embeds raw binary symbols, groups them into fixed-size patches, and generates plaintext as UTF-8 bytes.

The C5 embedding and patching pipeline is available in the project BLT module (Shankar, 2026e).

2 Dataset and Preprocessing

2.1 Dataset

The dataset consists of two line-aligned files containing 5,000 examples. The file `brown_cipher.txt` contains encrypted binary sequences, while `brown_plain.txt` contains the corresponding plaintext English sentences. Each ciphertext sequence is treated as the encoder input, and the plaintext on the same line is treated as the decoder target.

Exploratory data analysis confirmed that both files contain the same number of non-empty examples. The ciphertext contains only the binary symbols 0 and 1, whereas the plaintext contains 52 distinct uppercase and lowercase alphabetic characters. Table 1 summarizes the principal dataset statistics.

The ciphertext contains 13,665,487 occurrences

of 0 and 10,238,769 occurrences of 1. The considerably greater ciphertext lengths show that the encoder must process much longer sequences than the decoder generates. Alignment is preserved throughout preprocessing by storing each ciphertext and its corresponding plaintext as a single source–target example.

2.2 Data Splitting

The aligned examples are divided into training, validation, and test sets using a fixed random seed of 42. The split proportions are 70% for training, 10% for validation, and 20% for testing, corresponding to 3,500, 500, and 1,000 examples, respectively.

The dataset is split before tokenizer training. Consequently, the tokenizers learn their vocabularies and merge rules exclusively from the training data, preventing information from the validation and test sets from influencing model training.

2.3 Tokenization and Batch Construction

Configurations C1–C4 use two BPE-style tokenizers implemented from scratch. The source tokenizer learns variable-length binary subwords from the training ciphertext, while the target tokenizer learns subwords from the training plaintext. Beginning-of-sequence, end-of-sequence, padding, and unknown tokens are included in the tokenizer vocabularies.

BPE begins with individual characters and repeatedly merges the most frequent adjacent token pair until the required vocabulary size is reached. During batch construction, the learned tokenizers encode each source–target pair, construct the shifted decoder input and target sequences, pad the sequences, and generate Boolean padding masks.

Algorithm 1 summarizes the preprocessing and batch-construction pipeline used in the project.

For C5, the BPE stage is replaced by direct bit- and byte-level encoding. Source characters are converted into binary symbol IDs, while target sentences are converted into UTF-8 byte IDs. The resulting variable-length sequences are padded and masked using the same general batching procedure.

The preprocessing pipeline uses the default configuration values shown in Table 2. Source sequences are truncated to a maximum of 1,024 positions, while target sequences are limited to 256 positions. For C1–C4, separate source and target BPE tokenizers are trained with a maximum vocabulary size of 200 tokens each. Token ID 0 represents

Algorithm 1 Tokenization and batch construction

Require: Aligned dataset D , configuration C , random seed s

Ensure: Training, validation, and test batches

```

1:  $(D_{\text{train}}, D_{\text{val}}, D_{\text{test}})$   $\leftarrow$ 
   SPLITALIGNEDPAIRS( $D, s$ )
2: if  $C \in \{C1, C2, C3, C4\}$  then
3:    $T_{\text{src}} \leftarrow \text{TRAINBPE}(\text{ciphertext in } D_{\text{train}})$ 
4:    $T_{\text{tgt}} \leftarrow \text{TRAINBPE}(\text{plaintext in } D_{\text{train}})$ 
5:   for all  $(x, y)$  in a batch do
6:      $x_{\text{id}} \leftarrow \text{ENCODE}(T_{\text{src}}, x)$ 
7:      $y_{\text{id}} \leftarrow \text{ENCODE}(T_{\text{tgt}}, y)$ 
8:      $y_{\text{in}} \leftarrow [\text{BOS}] ++ y_{\text{id}}$ 
9:      $y_{\text{out}} \leftarrow y_{\text{id}} ++ [\text{EOS}]$ 
10:  end for
11: else  $\triangleright$  C5 byte-level pipeline
12:   for all  $(x, y)$  in a batch do
13:      $x_{\text{id}} \leftarrow \text{ENCODEBITS}(x)$ 
14:      $y_{\text{id}} \leftarrow \text{ENCODEUTFEIGHTBYTES}(y)$   $\leftarrow$ 
15:      $y_{\text{in}} \leftarrow [\text{BYTE\_BOS}] ++ y_{\text{id}}$ 
16:      $y_{\text{out}} \leftarrow y_{\text{id}} ++ [\text{BYTE\_EOS}]$ 
17:   end for
18: end if
19:  $X \leftarrow \text{PADBATCH}(x_{\text{id}})$ 
20:  $Y_{\text{in}} \leftarrow \text{PADBATCH}(y_{\text{in}})$ 
21:  $Y_{\text{out}} \leftarrow \text{PADBATCH}(y_{\text{out}})$ 
22:  $M_{\text{src}} \leftarrow (X = \text{PAD})$ 
23:  $M_{\text{tgt}} \leftarrow (Y_{\text{in}} = \text{PAD})$ 
24: return  $(X, M_{\text{src}}, Y_{\text{in}}, Y_{\text{out}}, M_{\text{tgt}})$ 
```

padding, token ID 1 represents BOS, and token ID 2 represents EOS.

The C5 configuration changes the tokenizer type from subword to b1t. It groups every 16 source-bit embeddings into a patch and uses a 32-dimensional local embedding for each bit before projecting the complete patch to the Transformer model dimension.

C5 does not use the 200-token BPE vocabulary. Its target vocabulary contains 256 possible UTF-8 byte values together with separate padding, BOS, and EOS tokens, resulting in a total target vocabulary size of 259.

3 Model Architecture

The model is a custom encoder–decoder Transformer implemented in PyTorch. The encoder converts the encrypted source sequence into contextual representations, while the decoder generates plain-

Parameter	Default	Purpose
max_src_len	1024	Maximum source length
max_target_len	256	Maximum target length
vocab_size	200	Maximum BPE vocabulary size
blt_patch_size	16	Source bits per C5 patch
blt_local_dim	32	Local C5 embedding size
src_pad_id	0	Source BPE padding ID
target_pad_id	0	Target BPE padding ID
bos_id	1	Beginning-of-sequence ID
eos_id	2	End-of-sequence ID
tokenizer_type	subword	Default input representation

Table 2: Default preprocessing and tokenization configuration. The BPE vocabulary and special-token IDs apply to C1–C4, while the BLT patch parameters apply to C5.

Config.	Position	Attention	Norm.	Input
C1	Sinusoidal	MHA	LayerNorm	Scratch BPE
C2	RoPE	MHA	LayerNorm	Scratch BPE
C3	Sinusoidal	GQA	LayerNorm	Scratch BPE
C4	Sinusoidal	MHA	RMSNorm	Scratch BPE
C5	Sinusoidal	MHA	LayerNorm	BLT-style

Table 3: Configurations used in the ablation study.

text autoregressively using masked self-attention and cross-attention over the encoder output. Figure 1 illustrates the complete architecture.

Each encoder layer contains self-attention and a position-wise feed-forward network. Each decoder layer contains masked self-attention, encoder–decoder cross-attention, and a feed-forward network. Residual connections, normalization, dropout, causal masks, and padding masks are applied throughout the architecture.

Scaled dot-product attention is computed as

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V, \quad (1)$$

where Q , K , and V are the query, key, and value tensors, d_k is the key dimension, and M is a causal or padding mask.

The default model uses a dimension of 128, four attention heads, two encoder layers, two decoder layers, a feed-forward dimension of 512, and dropout of 0.1. These settings are maintained across C1–C5 wherever possible to support a controlled comparison.

4 Experimental Configurations

Table 3 summarizes the five configurations used in the ablation study. Each configuration modifies one major component relative to the baseline wherever possible.

5 Training

All configurations were trained for 50 epochs using a batch size of 2 and a learning rate of 3×10^{-4} . Cross-entropy loss was used as the training objective, with padding positions excluded from the loss calculation.

Teacher forcing was applied during training by providing the shifted target sequence to the decoder. A causal mask prevented each target position from attending to future positions, while the source and target padding masks excluded padded elements from attention.

5.1 Hardware Environment

The experiments were conducted locally on an Apple Mac equipped with an M4 Pro processor and 24 GB of unified memory. PyTorch’s Metal Performance Shaders (MPS) backend was used to accelerate model training on the Apple GPU.

5.2 Memory Constraints

An initial architecture with eight encoder layers produced out-of-memory errors on the available hardware. Since self-attention has a memory complexity of $\mathcal{O}(BHS^2)$ and training must retain intermediate activations for backpropagation, the long source sequences substantially increased memory usage. The final model therefore used two encoder and two decoder layers with a batch size of 2, and these settings were kept consistent across all configurations.

5.3 Pipeline and Artifacts

The training pipeline calculates the training and validation losses for every epoch and stores the information required to reproduce and evaluate each experiment. A separate checkpoint directory is created for every configuration from C1 to C5.

The pipeline generates the following artifacts:

- **Model checkpoint:** The learned model parameters are saved in `model.pt`, allowing the trained model to be loaded for evaluation or inference.
- **Experiment configuration:** The preprocessing, architecture, and training parameters are stored in `config.json`.
- **Tokenizer state:** For configurations C1–C4, the learned source and target BPE vocabularies and merge rules are

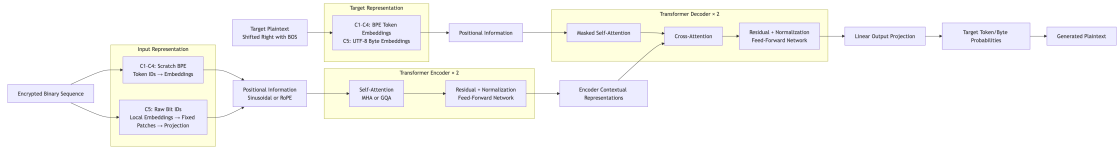


Figure 1: Architecture of the cipher-to-plaintext Transformer. C1–C4 use scratch BPE embeddings, while C5 uses local bit embeddings and fixed-size patches.

stored in `cipher_tokenizer.json` and `plain_tokenizer.json`, respectively.

- **Training history:** The training and validation losses recorded after every epoch are stored in `training_history.json`.
- **Final training metrics:** The final epoch, training loss, validation loss, and complete loss history are stored in `metrics.json`.
- **Training-loss visualization:** The file `training_loss.png` plots the training and validation losses across epochs, allowing convergence and possible overfitting to be examined.
- **Execution logs:** Separate log files record the progress and outcome of the training run for each experimental configuration.

Training and validation losses are also logged to Weights & Biases for experiment tracking and comparison. After training, each checkpoint directory is uploaded to Hugging Face, making the model parameters, tokenizer state, configuration, and training results available outside the local environment.

6 Evaluation

Greedy autoregressive decoding was used during inference. Starting with the BOS token, the decoder selects the highest-probability token at each step and appends it to the input for the next step. Generation stops when the EOS token is produced or the maximum target length is reached.

Each configuration was evaluated on 100 examples from the test split. The reported values are averages across these examples.

- **Bit-level accuracy:** Measures the proportion of matching bits after converting the predicted and target plaintext to their UTF-8 binary representations.

- **Exact-sequence accuracy:** Measures the proportion of predictions that exactly match their target sentences.
- **Levenshtein distance:** Measures the minimum number of character insertions, deletions, and substitutions required to transform a prediction into its target.
- **BLEU:** Measures word n -gram precision between the prediction and target, using up to four-grams and a brevity penalty.
- **ROUGE-1:** Measures unigram overlap between the prediction and target.
- **ROUGE-2:** Measures bigram overlap between the prediction and target.
- **ROUGE-L:** Measures overlap based on the longest common word subsequence.

Higher values indicate better performance for accuracy, BLEU, and ROUGE, whereas a lower Levenshtein distance indicates better reconstruction. All metrics are reported for the BPE-based configurations C1–C4. Since C5 generates UTF-8 bytes directly, it is evaluated using bit-level accuracy, exact-sequence accuracy, and Levenshtein distance.

6.1 Evaluation Artifacts

For each configuration, the evaluation pipeline saves:

- an aggregate evaluation summary;
- per-example predictions and metric values;
- an accuracy and text-overlap plot;
- a Levenshtein-distance distribution;
- a prediction-length comparison; and
- the corresponding training and validation loss curves.

Config.	Bit Acc.	Seq. Acc.	Lev. Dist.	ROUGE-L
C1	0.4363	0.00	504.97	0.1118
C2	0.4635	0.00	478.10	0.1388
C3	0.4417	0.00	494.04	0.1262
C4	0.4341	0.00	499.64	0.1245
C5	0.4134	0.03	436.16	—

Table 4: Reconstruction performance on 100 test examples. Higher accuracy and ROUGE-L values are better, while a lower Levenshtein distance is better.

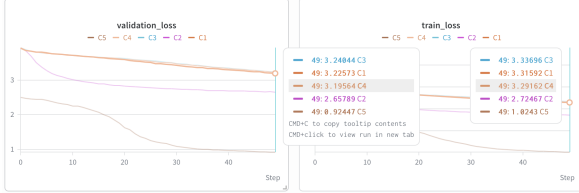


Figure 2: Training and validation loss for C1–C5 over 50 epochs.

A combined ablation summary and comparison plot are also generated for configurations C1–C5. Aggregate evaluation metrics are logged to Weights & Biases, while detailed predictions and visualizations are retained locally.

7 Results and Observations

Table 4 presents the reconstruction performance of the five configurations on 100 examples from the test split.

7.1 Training and Validation Loss

Figure 2 compares the training and validation losses of configurations C1–C5 over 50 epochs. The losses decrease across all configurations, indicating that each model learns from the training data.

Among the BPE-based models, C2 shows the greatest reduction in training and validation loss, reaching final values of approximately 2.72 and 2.66, respectively. C1, C3, and C4 converge to similar values, with final validation losses of approximately 3.23, 3.24, and 3.20.

C5 reaches the lowest numerical training and validation losses, approximately 1.02 and 0.92. However, C5 predicts UTF-8 bytes rather than BPE tokens, so its cross-entropy loss is calculated over a different output representation and should not be compared directly with the losses of C1–C4. The close relationship between the training and validation curves provides no strong indication of overfitting within the 50 training epochs.

The repeated utilization peaks show that the

training pipeline makes active use of the Apple GPU through the MPS backend. The fluctuations are consistent with alternating model computation, batch preparation, validation, logging, and checkpoint operations. GPU utilization alone does not represent memory consumption and therefore does not directly show the out-of-memory condition encountered with the earlier eight-layer architecture.

7.2 Effect of Positional Encoding

Among the BPE-based configurations, C2 achieves the strongest overall reconstruction performance. It obtains the highest bit-level accuracy of 0.4635, the highest ROUGE-L score of 0.1388, and the lowest Levenshtein distance among C1–C4.

C2 also obtains a noticeably lower training and validation loss than the sinusoidal baseline. These results suggest that RoPE is more effective than sinusoidal positional encoding for this task under the selected architecture and training conditions.

7.3 Effect of Attention

C3 achieves a slightly higher bit-level accuracy and ROUGE-L score than the C1 baseline. However, its Levenshtein distance and loss curves remain close to those of C1.

This indicates that grouped-query attention provides performance comparable to multi-head attention while using fewer independent key and value projections. It does not, however, produce a substantial improvement in reconstruction quality in the present experiment.

7.4 Effect of Normalization

C4 obtains a slightly lower bit-level accuracy than C1 but produces a modestly higher ROUGE-L score and a lower Levenshtein distance. Its training and validation loss curves remain close to those of the baseline.

Replacing LayerNorm with RMSNorm therefore does not provide a clear overall advantage, although it preserves performance close to the baseline using a simpler normalization operation.

7.5 Byte-Level Configuration

C5 is the only configuration to achieve non-zero exact-sequence accuracy, correctly reconstructing 3% of the evaluated examples. It also obtains the lowest average Levenshtein distance, although its bit-level accuracy is lower than that of the BPE-based configurations.

Config.	Target and prediction	ED
C1	T: Flower Fairies of the Autumn Blackie P: Flower Fairies of the Blackie	7
C2	T: Flower Fairies of the Autumn Blackie P: Flower Fairies of the Fairies Blackie	7
C3	T: Flower Fairies of the Autumn Blackie P: Flower Fairies of the Ward War I Blackie	10
C4	T: Flower Fairies of the Autumn Blackie P: Flower Fairies of the Flower Fairies of the Fairies . . .	172
C5	T: The growth of Bintulu s population is shown below P: The growth of Bintulu s population is shown below	0

Table 5: Representative predictions from C1–C5. ED denotes character-level edit distance; the repetitive C4 output is abbreviated.

These results indicate that the byte-level representation can generate some complete reconstructions, but its improvements are not consistent across all metrics. Since C5 predicts UTF-8 bytes instead of BPE tokens, its loss values and token-level behaviour are not directly comparable with those of C1–C4.

7.6 Qualitative Prediction Examples

To complement the aggregate metrics, the generated predictions were examined qualitatively. Table 5 presents representative exact and near-correct outputs selected from the per-example prediction files.

C5 exactly reconstructs three of the 100 evaluated target sequences, consistent with its exact-sequence accuracy of 0.03. It also produces two near-exact predictions that differ from their targets only through additional characters at the end of the sequence.

The BPE-based configurations do not produce any exact matches in the evaluated sample. However, C1, C2, and C3 correctly reconstruct substantial portions of the short “Flower Fairies” example. Their remaining errors involve the deletion or substitution of a small phrase, showing that the models can recover parts of shorter sequences even when the complete prediction is not exact.

These examples also suggest that reconstruction quality is sensitive to sequence length. The most accurate outputs are short sentences, whereas longer predictions frequently contain repeated words or phrases. This behaviour is consistent with error accumulation during greedy autoregressive decoding.

8 Conclusion

The experiments show that all five configurations learn aspects of the ciphertext-to-plaintext mapping.

Nevertheless, complete reconstruction remains difficult. Exact-sequence accuracy is zero for C1–C4, and the BLEU and ROUGE scores remain low.

One likely limitation is the relatively small model size used to process long source sequences. Binary ciphertext sequences may contain substantially more positions than the corresponding plaintext sentences. This increases the difficulty of learning long-range relationships between source bits and target tokens.

Greedy decoding may also limit output quality because it selects the highest-probability token at each step without considering alternative sequences. Beam-search decoding could potentially improve reconstruction by maintaining multiple candidate outputs.

The comparison should additionally be interpreted with caution because the current evaluation uses 100 test examples rather than the entire test split.

Future work could explore larger models, longer training, improved hyperparameter selection, learning-rate scheduling, regularization, beam-search decoding, and evaluation on the complete test set.

References

- Bhavani Shankar. 2026a. Implementation of layer normalization and rms normalization. https://github.com/ybshankar010/anlp_transformer/blob/main/src/models/norm.py. Source code, accessed September 1, 2026.
- Bhavani Shankar. 2026b. Implementation of multi-head and grouped-query attention. https://github.com/ybshankar010/anlp_transformer/blob/main/src/models/attention.py. Source code accessed August 31, 2026.
- Bhavani Shankar. 2026c. Implementation of sinusoidal and rotary positional encoding. https://github.com/ybshankar010/anlp_transformer/blob/main/src/models/positional.py. Source code, accessed September 1, 2026.
- Bhavani Shankar. 2026d. Rotary positional encoding implementation. https://github.com/ybshankar010/anlp_transformer/blob/main/src/models/attention.py#L48. Source code, accessed September 1, 2026.
- Bhavani Shankar. 2026e. Simplified byte latent transformer-style implementation. https://github.com/ybshankar010/anlp_transformer/blob/main/src/models/blt.py. Source code, accessed September 1, 2026.